

Nomad ECS, UI, and State Machine Blueprint

This file is the naming plan for Nomad before implementation. It lists the files, entity names, component names, system names, UI folders, game states, and how each part should be used.

Core Folder Structure

```
C2_ADA Shared/  
  ECS/  
    EntityManager.swift  
    NomadECSCatalog.swift  
  
    Entities/  
      PlayerEntity.swift  
      VehicleEntity.swift  
      ObstacleEntity.swift  
      BackgroundEntity.swift  
      EffectEntity.swift  
      ParticleEntity.swift  
  
    Components/  
      NodeComponent.swift  
      AssetComponent.swift  
      TransformComponent.swift  
      MovementComponent.swift  
      VelocityComponent.swift  
      InputIntentComponent.swift  
      RideAttachmentComponent.swift  
      LaunchComponent.swift  
      CollisionComponent.swift  
      StateComponent.swift  
      VehicleComponent.swift  
      ObstacleComponent.swift  
      EffectComponent.swift  
      LifetimeComponent.swift  
      ScoreComponent.swift  
      SpawnComponent.swift  
  
    Systems/  
      InputSystem.swift  
      MovementSystem.swift  
      LaunchSystem.swift  
      LatchSystem.swift  
      CollisionSystem.swift  
      SpawnSystem.swift  
      ScoreSystem.swift  
      StateMachineSystem.swift  
      VehicleTimerSystem.swift  
      AnimationSystem.swift  
      EffectSystem.swift  
      CleanupSystem.swift  
  
    Helper/  
      GameConfiguration.swift  
      InputPhase.swift  
      GameplayRule.swift  
      NomadAsset.swift  
      EntityKind.swift  
  
    State/  
      GameState.swift  
      PlayerState.swift  
      VehicleState.swift  
      ObstacleState.swift
```

```

ScreenState.swift
StateMachine.swift

UI/
Nodes/
    NomadNodeFactory.swift
    GameHUDNode.swift
    SettingsPanelNode.swift
    PausePanelNode.swift
    GameOverPanelNode.swift
    TouchPromptNode.swift
    DistanceCounterNode.swift

Screens/
    HomeScreenNode.swift
    SettingsScreenNode.swift
    PlayScreenNode.swift
    PauseScreenNode.swift
    FailScreenNode.swift
    NewHighscoreScreenNode.swift

Scenes/
    HomeScene.swift
    PlayScene.swift
    NomadSceneFactory.swift

```

ECS Usage

`EntityManager.swift` should be the single place that owns active gameplay entities. `GameScene` asks it to add, remove, update, and query entities instead of keeping many separate arrays.

Recommended `EntityManager` responsibilities:

```

add(entity)
remove(entity)
removeAll()
update(deltaTime)
entities(with componentType)
firstEntity(with componentType)
addNode(for entity, to scene)
removeNode(for entity)

```

Recommended system update order:

```

InputSystem
StateMachineSystem
MovementSystem
VehicleTimerSystem
LaunchSystem
LatchSystem
CollisionSystem
SpawnSystem
ScoreSystem
AnimationSystem
EffectSystem
CleanupSystem

```

Entities

Entity	File	Usage
PlayerEntity	ECS/Entities/PlayerEntity.swift	The red player/rider. Holds player state, input intent, latch data, launch data, collision, and visible node.
VehicleEntity	ECS/Entities/VehicleEntity.swift	Jeep, van, pickup, or monster truck. Holds movement, vehicle type, warning timer, collision, and visual pose.
ObstacleEntity	ECS/Entities/ObstacleEntity.swift	Rock, desert tree, or cactus that can end the run when hit.
BackgroundEntity	ECS/Entities/BackgroundEntity.swift	Floor, left wall, right wall, road, and scrolling background pieces.
EffectEntity	ECS/Entities/EffectEntity.swift	One-shot effects such as sand splash and smoke.
ParticleEntity	ECS/Entities/ParticleEntity.swift	Small repeated particles such as sand particles behind movement or impact.

Components

Component	Usage
NodeComponent	Stores the SKNode for a GameplayKit entity.
AssetComponent	Stores which asset name or sprite variant the entity should show.
TransformComponent	Stores position, scale, rotation, and z position when you want logic separated from the node.
MovementComponent	Stores road lane position, drag start, drag offset, and clamped movement.
VelocityComponent	Stores velocity and acceleration for moving objects.
InputIntentComponent	Stores current input: idle, holding, dragging, released.
RideAttachmentComponent	Stores which vehicle the player is currently attached to.
LaunchComponent	Stores jump velocity, gravity, airborne position, and whether the player is in the air.
CollisionComponent	Stores hitbox size, category, and collision enable/disable state.
StateComponent	Stores entity state machines, such as PlayerState or VehicleState.
VehicleComponent	Stores vehicle kind, vehicle pose, ride timer, and warning/scolding timing.
ObstacleComponent	Stores obstacle kind and whether it is active, passed, or collided.
EffectComponent	Stores effect kind and playback state.
LifetimeComponent	Counts down temporary entities, then lets cleanup remove them.
ScoreComponent	Stores distance score, high score check, and run result.
SpawnComponent	Stores spawn timing, spawn distance, and next entity type.

Systems

System	Usage
InputSystem	Converts touch hold, drag, and release into InputPhase.
MovementSystem	Moves the current vehicle left and right while the player is holding/dragging.
LaunchSystem	Launches the player from a vehicle and applies gravity while airborne.
LatchSystem	Checks if the airborne player intersects a new vehicle and attaches them.
CollisionSystem	Checks player/vehicle collisions with obstacles and ground.
SpawnSystem	Spawns next vehicles, obstacles, background pieces, and effects.
ScoreSystem	Tracks distance and decides normal fail vs new high score.
StateMachineSystem	Applies allowed state transitions for game, screen, player, vehicle, obstacle, and effects.
VehicleTimerSystem	Handles ride timer rules: normal, person pops out, hitting states, forced jump.
AnimationSystem	Switches visible assets based on state, such as player legs left/right or vehicle hitting poses.
EffectSystem	Plays sand splash, smoke, and sand particles.
CleanupSystem	Removes expired effects, particles, and off-screen entities.

Asset Names

Create these image sets later in `Assets.xcassets` so code can reference them by exact name.

Player

```
PlayerLegsStraight  
PlayerLegsLeft  
PlayerLegsRight
```

Jeep

```
JeepNormal  
JeepPersonPoppingOut  
JeepPersonHitting1  
JeepPersonHitting2
```

Vehicle Variants

```
VanNormal  
VanPersonPoppingOut  
VanPersonHitting1
```

VanPersonHitting2

PickupNormal
PickupPersonPoppingOut
PickupPersonHitting1
PickupPersonHitting2

MonsterTruckNormal
MonsterTruckPersonPoppingOut
MonsterTruckPersonHitting1
MonsterTruckPersonHitting2

Obstacles

Rock
DesertTree
Cactus

Background

Floor
LeftWall
RightWall

Effects

SandSplash
Smoke
SandParticle

State Names

ScreenState

Controls which UI screen/page is visible.

home
settings
play
paused
fail
newHighscore

Transitions:

home -> settings
settings -> home
home -> play
play -> paused
paused -> play
paused -> home
play -> fail
play -> newHighscore
fail -> play
newHighscore -> play
fail -> home
newHighscore -> home

GameState

Controls the main gameplay condition.

```
waitingToStart
riding
vehicleWarning
jumping
latching
falling
gameOver
paused
```

Transitions:

```
waitingToStart -> riding
riding -> vehicleWarning
vehicleWarning -> riding
riding -> jumping
vehicleWarning -> jumping
jumping -> latching
latching -> riding
jumping -> falling
falling -> gameOver
riding -> gameOver
gameOver -> waitingToStart
riding -> paused
paused -> riding
```

PlayerState

Controls the player/rider.

```
idleOnVehicle
draggingLeft
draggingRight
scolded
jumping
latching
falling
crashed
```

Usage:

```
idleOnVehicle: player is attached to a vehicle and not dragging.
draggingLeft: hold and drag left.
draggingRight: hold and drag right.
scolded: player has stayed on the vehicle too long and the NPC reacts.
jumping: player released hold or was forced into the air.
latching: player is trying to attach to the next vehicle.
falling: player missed the latch window.
crashed: player hit an obstacle or fell.
```

VehicleState

Controls vehicles and the "diomelin" warning sequence.

```
normal
controlled
personPoppingOut
personHitting1
personHitting2
```

- forcedLaunch
- latchTarget
- latched
- crashed
- despawned

Suggested timer:

- 0s: normal
- 3s: personPoppingOut
- 5s: personHitting1
- 6s: personHitting2
- 7s: forcedLaunch

InputState

Controls touch logic.

- idle
- holding
- draggingLeft
- draggingRight
- released
- cancelled

ObstacleState

Controls obstacle lifetime.

- spawning
- active
- passed
- collided
- despawned

EffectState

Controls temporary visual effects.

- inactive
- playing
- finished

Gameplay Rules

Rule 1: Start

When the play screen opens:

- Screen does not move.
- Prompt says hold to start.
- Player starts attached to a standard vehicle in the middle lane.

State change:

```
ScreenState.play
GameState.waitingToStart
PlayerState.idleOnVehicle
VehicleState.normal
```

Rule 2: Hold and Drag

When player holds the screen:

```
InputState.holding
GameState.playing
VehicleState.controlled
```

When player drags left:

```
InputState.draggingLeft
PlayerState.draggingLeft
MovementSystem moves vehicle left.
AnimationSystem shows PlayerLegsLeft.
```

When player drags right:

```
InputState.draggingRight
PlayerState.draggingRight
MovementSystem moves vehicle right.
AnimationSystem shows PlayerLegsRight.
```

Rule 3: Vehicle Warning / Player Diomelin

When player stays on one vehicle too long:

```
VehicleTimerSystem counts ride time.
At 3s, VehicleState.personPoppingOut.
At 5s, VehicleState.personHitting1.
At 6s, VehicleState.personHitting2.
At 7s, VehicleState.forcedLaunch and PlayerState.jumping.
```

Rule 4: Release to Jump

When player releases hold:

```
InputState.released
GameState.playing
PlayerState.jumping
LaunchSystem detaches player and moves them through the air.
```

Rule 5: Hold While Airborne to Latch

When player is airborne and holds:

```
LatchSystem checks player hitbox against nearby vehicle hitboxes.
If intersecting, PlayerState.latching -> PlayerState.idleOnVehicle.
If not intersecting, PlayerState.falling -> GameState.gameOver.
```


Rule 6: Obstacle Collision

When vehicle/player collides with obstacle:

```
CollisionSystem detects vehicle or player against ObstacleEntity.  
ObstacleState.collided  
PlayerState.crashed  
VehicleState.crashed  
GameState.gameOver
```

Rule 7: Fail vs New High Score

When game over happens:

```
ScoreSystem compares current distance with saved high score.  
If current distance is higher, ScreenState.newHighscore.  
Otherwise, ScreenState.fail.
```

UI Usage

UI should mostly be SpriteKit nodes and scenes, not ECS entities, unless the thing affects gameplay.

UI File	Usage
NomadNodeFactory.swift	Shared factory for player, vehicle, obstacle, effect, labels, and buttons.
GameHUDNode.swift	Shows distance, pause button, and run timer if needed.
SettingsPanelNode.swift	Music and haptics toggles.
PausePanelNode.swift	Resume, home, and settings buttons.
GameOverPanelNode.swift	Fail or new high score result.
TouchPromptNode.swift	"Hold to Start" and other short prompts.
DistanceCounterNode.swift	Reusable distance label.
HomeScreenNode.swift	Title, high score, start button, settings button.
SettingsScreenNode.swift	Settings overlay/screen.
PlayScreenNode.swift	Gameplay HUD and touch prompt layer.
PauseScreenNode.swift	Pause overlay.
FailScreenNode.swift	Normal game over page.
NewHighscoreScreenNode.swift	Crown/high score game over page.
HomeScene.swift	SpriteKit scene for home/settings flow.
PlayScene.swift	SpriteKit gameplay scene, or your existing GameScene.
NomadSceneFactory.swift	Creates and switches between home/play scenes.

Implementation Order

1. Keep current `PlayerEntity`, `VehicleEntity`, `InputSystem`, `MovementSystem`, `LaunchSystem`, and `LatchSystem`.
2. Add `EntityManager.swift`.
3. Add state models and `StateMachine.swift`.
4. Add collision components and `CollisionSystem.swift`.
5. Add `VehicleTimerSystem.swift` for the warning/diomelin sequence.
6. Add UI nodes/screens after gameplay states are stable.
7. Replace placeholder shapes with real assets using the asset names above.